

Lecture 16 – Deep Learning: Training a Neural Network

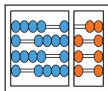
[ISLP 10.6–10.8]

MC886 – Machine Learning

Marcelo S. Reis [†]

[†] Departamento de Sistemas de Informação
Instituto de Computação, Universidade Estadual de Campinas

Campinas, April 27, 2026



**Instituto de
Computação**

UNIVERSIDADE ESTADUAL DE CAMPINAS



UNICAMP

Today's summary

When to use deep learning

The backpropagation algorithm

Regularization and stochastic gradient descent

Interpolation and double descent

When to use deep learning?

- ❖ Should we use deep learning on every problem?
- ❖ Case study on **Hitters** data
 - Regression of salary on 19 variables
 - Data of 176 players for training, 87 for test
 - Three approaches: Linear regression, LASSO and neural network with single hidden-layer and 64 ReLU units

Prediction results on **Hitters** test data

Model	# Parameters	Mean Abs. Error	Test Set R^2
Linear Regression	20	254.7	0.56
LASSO	12	252.3	0.51
Neural Network	1345	257.4	0.54

Key considerations:

1. Prediction performance
2. Computational requirements
3. Model interpretability

Fitting a neural network

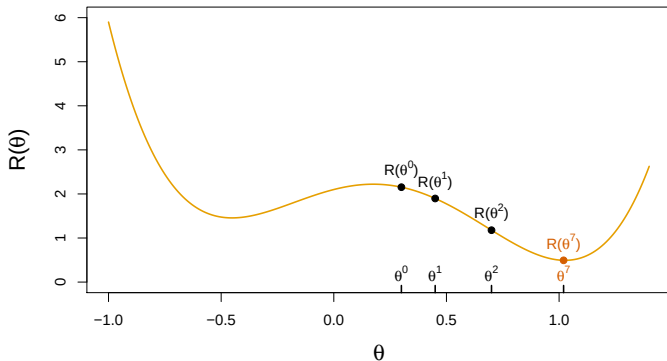
- ❖ Consider a neural network f_θ with parameters θ
- ❖ For a squared error loss, the optimization objective can be written as:

$$R(\theta) = \frac{1}{2} \sum_{i=1}^n (y_i - f_\theta(x_i))^2, \quad (10.25)$$

where $(x_1, y_1), \dots, (x_n, y_n)$ are the training observations

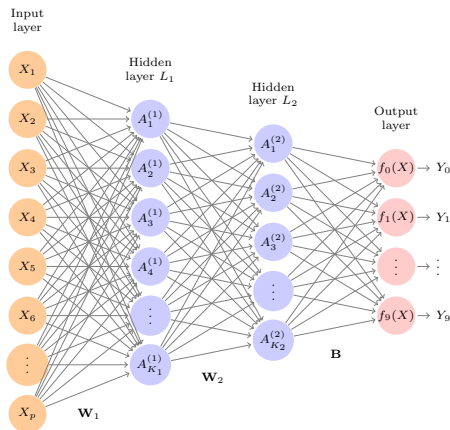
- ❖ We use the *gradient descent algorithm* to adjust θ with the training dataset

Example of gradient descent for unidimensional θ

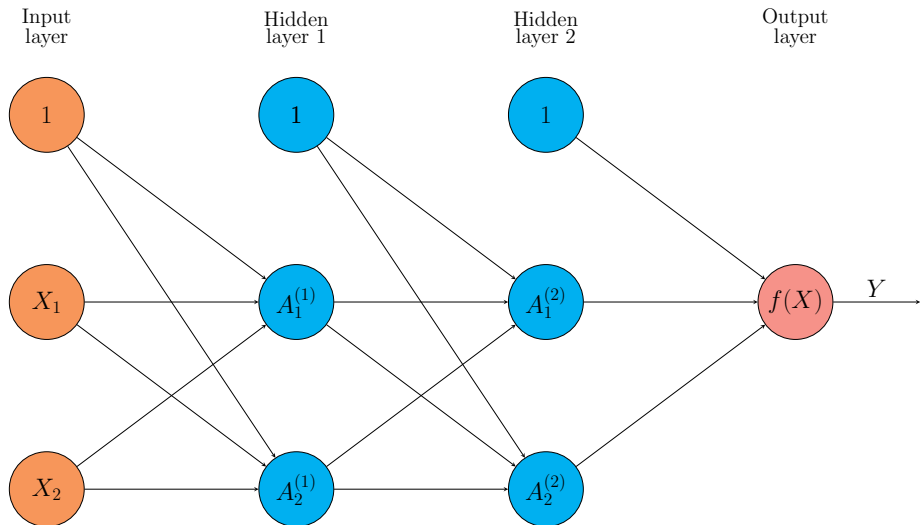


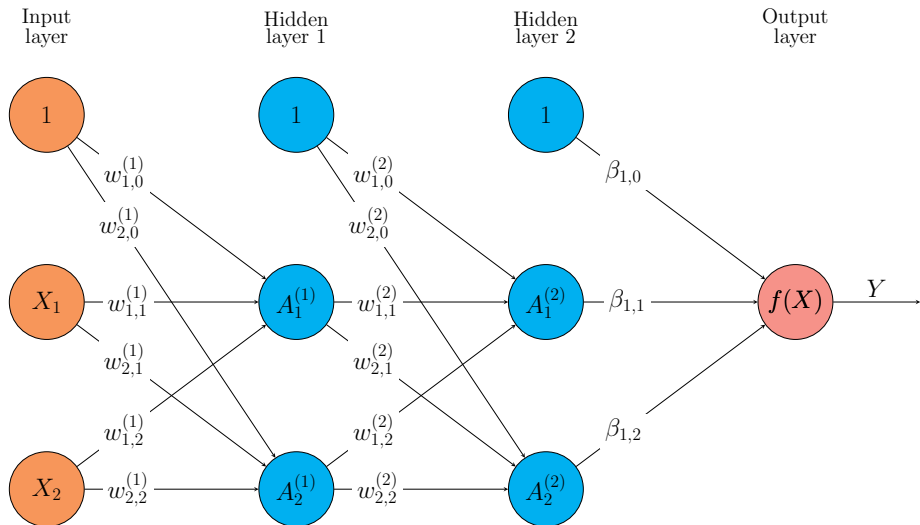
θ^t is the parameter value at the t -th iteration

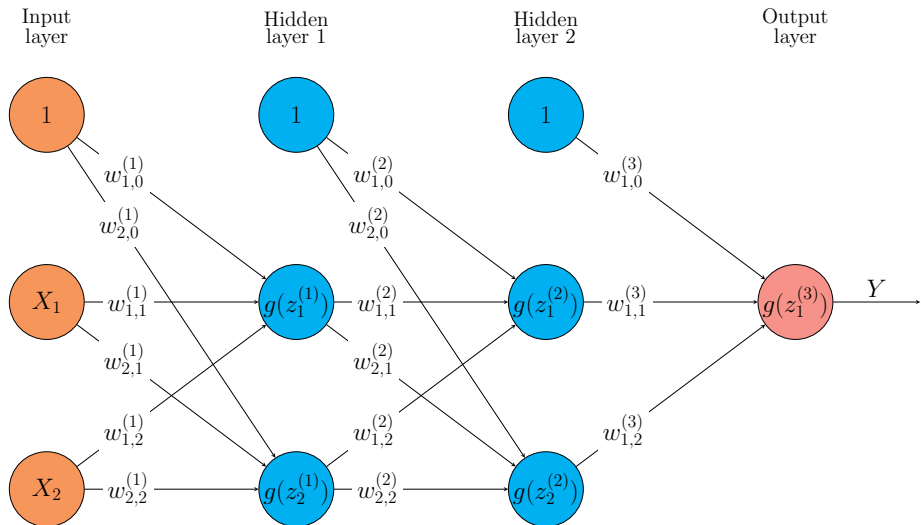
Multilayer network seen some lectures ago



- ❖ We'll work on a simpler version of it, with:
 - 2 variables in the input layer (X_1 and X_2)
 - 2 hidden layers with 2 units each
 - output layer with a single unit







$$z_k^{(\ell)} = w_{k0}^{(\ell)} + \sum_{j=1}^{K_{\ell-1}} w_{kj}^{(\ell)} A_j^{(\ell-1)}, \text{ where } A_j^{(\ell-1)} = \begin{cases} g(z_j^{(\ell-1)}), & \text{if } \ell > 1 \\ X_j, & \text{otherwise} \end{cases}$$

Weights in a feedforward neural network

A set θ of weights from L matrices $\mathbf{W}^{(\ell)}$
(to simplify our analysis, from now on $\beta_{kj} \equiv w_{kj}^{(L)}$, so the weights of the output layer compose the matrix $\mathbf{W}^{(L)}$)

For each entry $w_{kj}^{(\ell)}$ in $\mathbf{W}^{(\ell)}$:

- ℓ is the layer index
- k is the index of a unit at layer ℓ (which has K_ℓ units)
- j is the index of a unit (or input) j at layer $\ell - 1$

❖ $|\theta|$ is the total number of weights in the network

Computing gradient for the weights

- ❖ We need to compute $\nabla R(\theta) = \frac{\partial R(\theta)}{\partial \theta} \Big|_{\theta=\theta^t}$
- ❖ Computationally expensive when either the dataset size (n) and the number of weights ($|\theta|$) are large
- ❖ Large n : each iteration of gradient descent requires the usage of all points (a problem already seen for linear models)
- ❖ Large $|\theta|$: we must adopt one of two costly alternatives:
 - To numerically estimate of the gradient
 - To unroll $f_{\theta}(X)$

Workaround for the large n

- ❖ Instead, we compute $\nabla e(\theta) = \frac{\partial e(\theta)}{\partial w_{kj}^{(\ell)}}$, where e is a pointwise error function

- Example, the squared error: $e(\theta) = (Y - f_{\theta}(X))^2$

- ❖ Rational: $E[e(\theta)] = \frac{1}{n} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2 = R(\theta)$

- ❖ Equivalent to a mini-batch procedure with batch size equal to one (larger mini-batches might yield better estimates of $R(\theta)$)

Numerical estimation of the gradient

- ❖ Suppose the weight $w_{kj}^{(\ell)}$ has value ω for the error $e(\theta)$
- ❖ We can choose a small $\epsilon > 0$ (e.g. 10^{-4}) and estimate the gradient $\frac{\partial e(\theta)}{\partial w_{kj}^{(\ell)}}$ by computing two quantities:
 - $e(\theta_{+\epsilon})$, where $w_{kj}^{(\ell)} := \omega + \epsilon$
 - $e(\theta_{-\epsilon})$, where $w_{kj}^{(\ell)} := \omega - \epsilon$
- ❖ The gradient estimate is then given by:

$$\frac{e(\theta_{+\epsilon}) - e(\theta_{-\epsilon})}{\epsilon - (-\epsilon)} = \frac{e(\theta_{+\epsilon}) - e(\theta_{-\epsilon})}{2\epsilon}$$

Unrolling $f_{\theta}(X)$

- ❖ Suppose we want to compute of the pointwise squared error:

$$e(\theta) = (Y - f_{\theta}(X))^2$$

- ❖ Consider the weight $w_{1,1}^{(1)}$ of the network in the previous figure
- ❖ To compute $\frac{\partial e(\theta)}{\partial w_{1,1}^{(1)}}$, we must “unroll” $e(\theta)$ to get an expression that is a function of $w_{1,1}^{(1)}$ and then compute the derivative of $e(\theta)$ with respect to $w_{1,1}^{(1)}$

Unrolling $f_\theta(X)$ to compute $\frac{\partial e(\theta)}{\partial w_{1,1}^{(1)}}$

$$\begin{aligned}e(\theta) &= (Y - f_\theta(X))^2 \\&= (Y - A_1^{(3)})^2 \\&= (Y - g(w_{1,0}^{(3)} + w_{1,1}^{(3)} A_1^{(2)} + w_{1,2}^{(3)} A_2^{(2)}))^2 \\&= (Y - g(w_{1,0}^{(3)} + w_{1,1}^{(3)} g(w_{1,0}^{(2)} + w_{1,1}^{(2)} A_1^{(1)} + w_{1,2}^{(2)} A_2^{(1)}) + w_{2,2}^{(3)} A_2^{(2)}))^2 \\&= (Y - g(w_{1,0}^{(3)} + w_{1,1}^{(3)} g(w_{1,0}^{(2)} + w_{1,1}^{(2)} g(w_{1,0}^{(1)} + w_{1,1}^{(1)} X_1 + w_{1,2}^{(1)} X_2) + \\&\quad + w_{1,2}^{(2)} A_2^{(1)}) + w_{2,2}^{(3)} A_2^{(2)}))^2\end{aligned}$$

- ❖ We must compute the partial derivative of this expression and repeat this procedure for all other $w_{kj}^{(\ell)}$!!!
- ❖ Let's see a more efficient way to compute those derivatives

..but first, a review: Chain rule

- ❖ If a variable z depends on a variable y , and y depends on a variable x , then z depends on x as well. The **chain rule** is expressed as:

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$

- ❖ Example:

$$z = 4x^2 + 4x + 1$$

$$y = 2x + 1$$

- ❖ Clearly, $\frac{dz}{dx} = 8x + 4$

Chain rule

❖ Moreover, observe that:

$$4x^2 + 4x + 1 = (2x + 1)^2 = y^2$$

❖ Thus, let us inspect now $\frac{dz}{dy} \frac{dy}{dx}$:

$$\begin{aligned}\frac{dz}{dy} \frac{dy}{dx} &= \frac{d(y^2)}{dy} \frac{d(2x + 1)}{dx} \\ &= (2y) (2 + 0) \\ &= (2(2x + 1)) (2 + 0) \\ &= 8x + 4.\end{aligned}$$

Efficient computation of the gradient

- ❖ We need an alternative to the two previous approaches to compute $\nabla e(\theta) = \frac{\partial e(\theta)}{\partial w_{kj}^{(\ell)}}$ for all j, k, ℓ

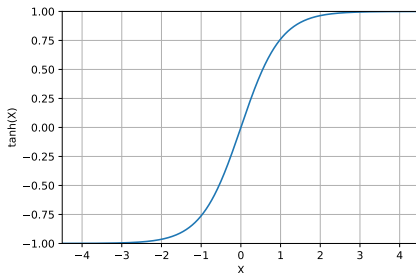
- ❖ The trick is the chain rule:
$$\frac{\partial e(\theta)}{\partial w_{kj}^{(\ell)}} = \underbrace{\frac{\partial e(\theta)}{\partial z_k^{(\ell)}}}_{\delta_k^{(\ell)}} \times \underbrace{\frac{\partial z_k^{(\ell)}}{\partial w_{kj}^{(\ell)}}}_{A_j^{(\ell-1)}}$$

- ❖ Two-step procedure: we compute $A_j^{(\ell-1)}$ forward

- ❖ Then we can compute $\delta_k^{(\ell)}$ backwards

Before we start...

...we'll adopt the hyperbolic tangent ($\tanh$) as the activation function:



- ❖ Differentiable function (gradient descent-friendly)

- ❖ Its derivative is easy to compute ($\tanh'(x) = 1 - \tanh^2(x)$)

Backpropagation for $\ell = L$ (with $\tanh$ as g)

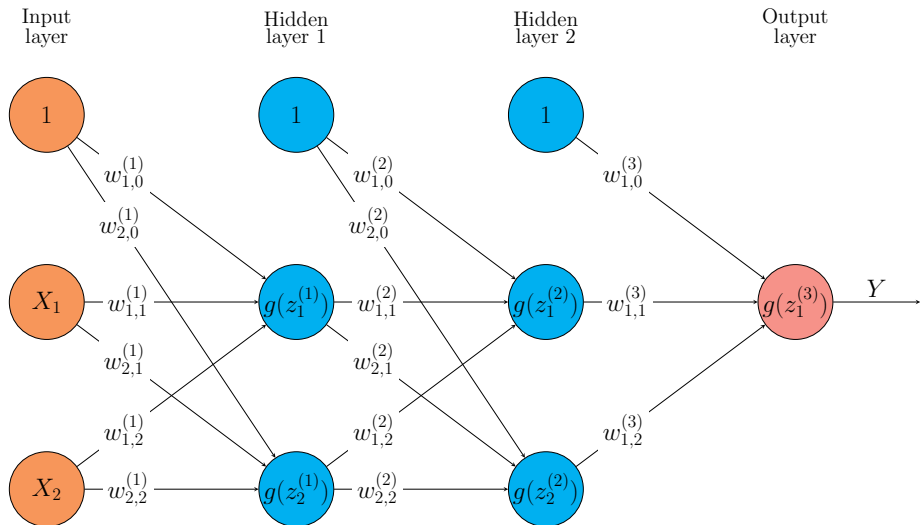
Let's also define $e(\cdot)$ as the quadratic error and compute $\delta_1^{(L)}$ as a function of the observation $(\mathbf{x}, y)$

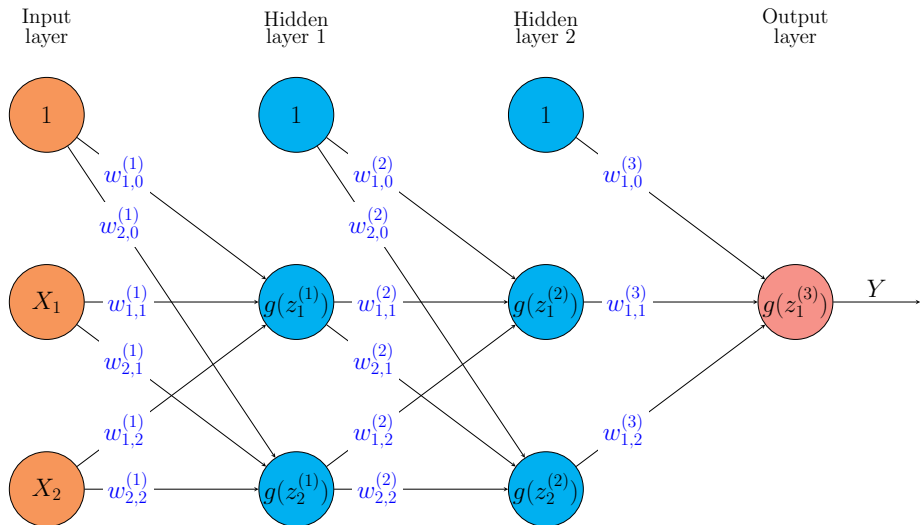
$$\begin{aligned}\delta_1^{(L)} &= \frac{\partial e(\theta)}{\partial z_1^{(L)}} \\&= \frac{\partial}{\partial z_1^{(L)}} (y - f(\mathbf{x}))^2 \\&= \frac{\partial}{\partial z_1^{(L)}} (y - A_1^{(L)})^2 \\&= \frac{\partial}{\partial z_1^{(L)}} (y - g(z_1^{(L)}))^2 \\&= 2(y - g(z_1^{(L)}))(1 - g^2(z_1^{(L)})) \quad (\text{chain rule and } \tanh')\end{aligned}$$

Backpropagation for $1 \leq \ell - 1 < L$ (with $\tanh$ as g)

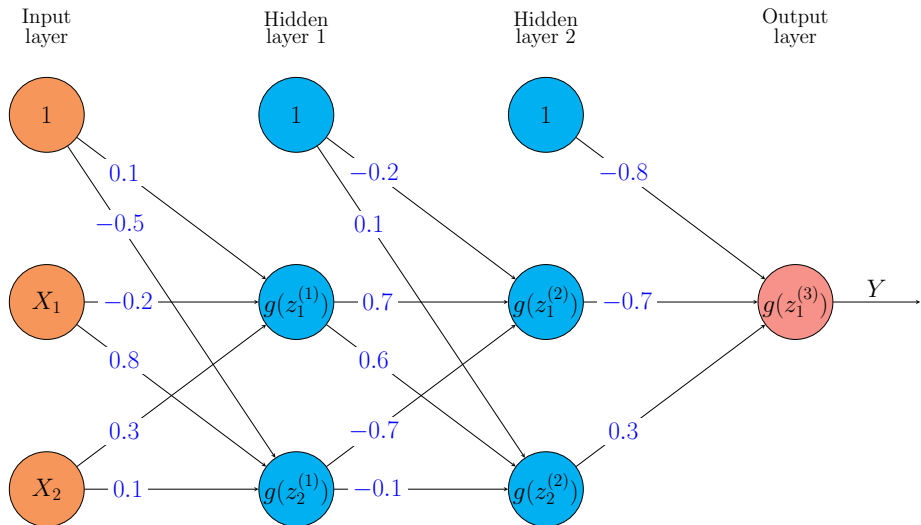
$$\begin{aligned}\delta_j^{(\ell-1)} &= \frac{\partial e(\theta)}{\partial z_j^{(\ell-1)}} \\&= \sum_{k=1}^{K_\ell} \frac{\partial e(\theta)}{\partial z_k^{(\ell)}} \times \frac{\partial z_k^{(\ell)}}{\partial A_j^{(\ell-1)}} \times \frac{\partial A_j^{(\ell-1)}}{\partial z_j^{(\ell-1)}} \\&= \sum_{k=1}^{K_\ell} \delta_k^{(\ell)} \times \frac{\partial (w_{k0}^{(\ell)} + \sum_{j'=1}^{K_{\ell-1}} w_{kj'}^{(\ell)} A_{j'}^{(\ell-1)})}{\partial A_j^{(\ell-1)}} \times \frac{\partial \tanh(z_j^{(\ell-1)})}{\partial z_j^{(\ell-1)}} \\&= \sum_{k=1}^{K_\ell} \delta_k^{(\ell)} \times w_{kj}^{(\ell)} \times (1 - \tanh^2(z_j^{(\ell-1)})) \\&= (1 - (A_j^{(\ell-1)})^2) \sum_{k=1}^{K_\ell} \delta_k^{(\ell)} \times w_{kj}^{(\ell)}\end{aligned}$$

Simulation of BACKPROPAGATION

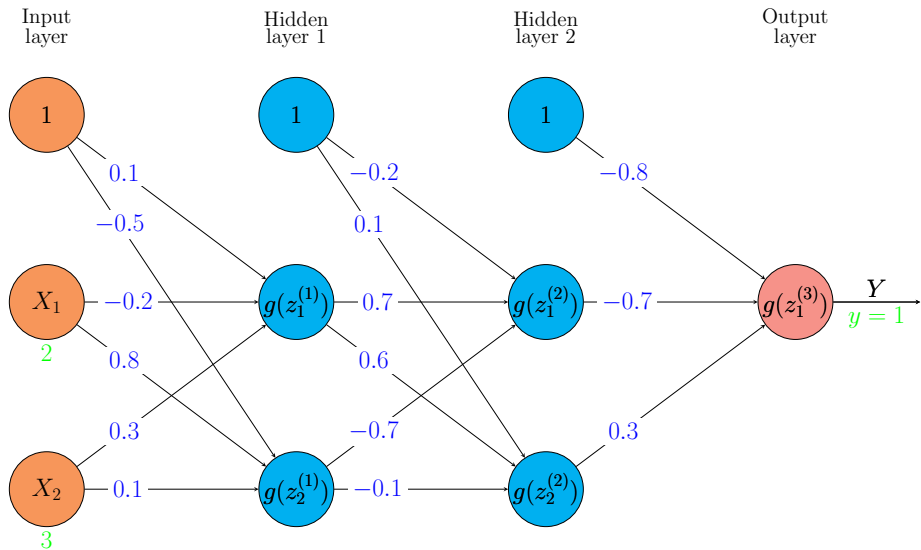




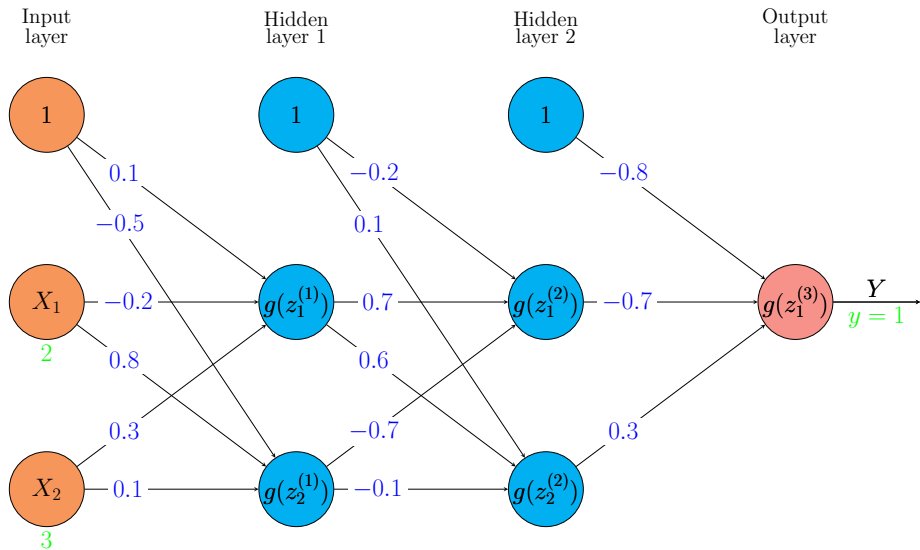
Initializing $w_{kj}^{(\ell)}$ with a value in $[-1, 1]$



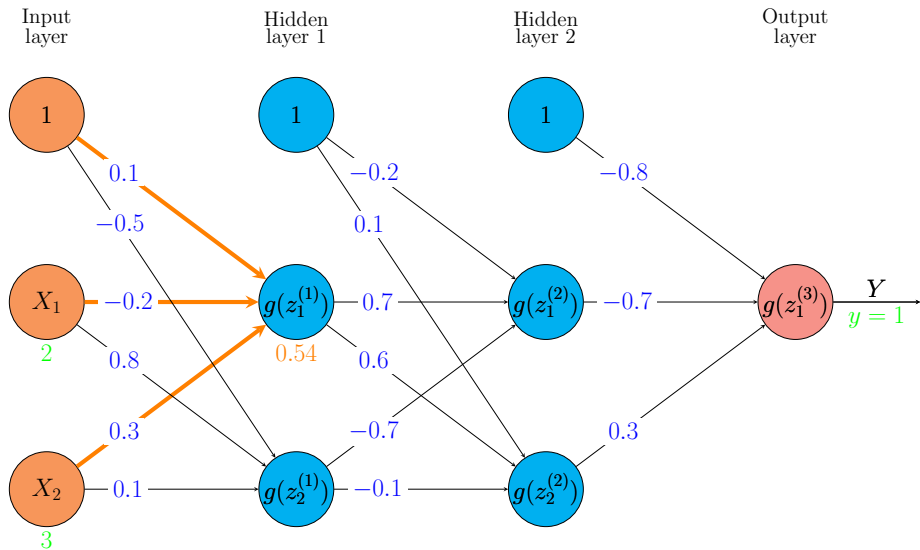
Initializing $w_{kj}^{(\ell)}$ with a value in $[-1, 1]$



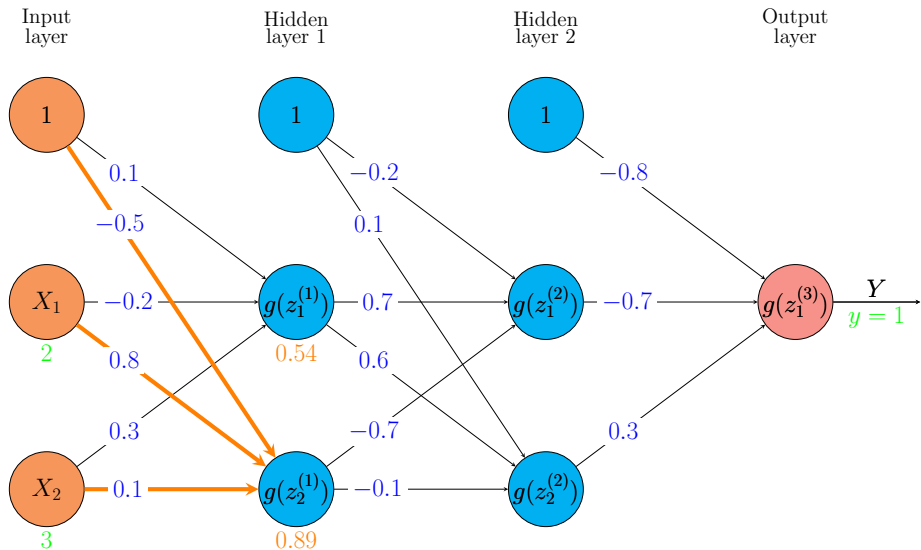
Inputting the observation $\left(\mathbf{x} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, y = 1 \right)$



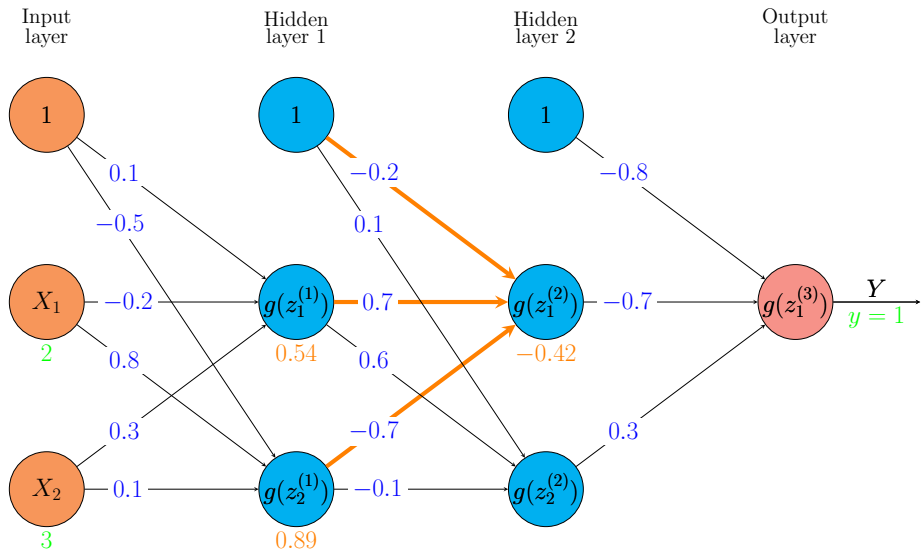
Forward propagation



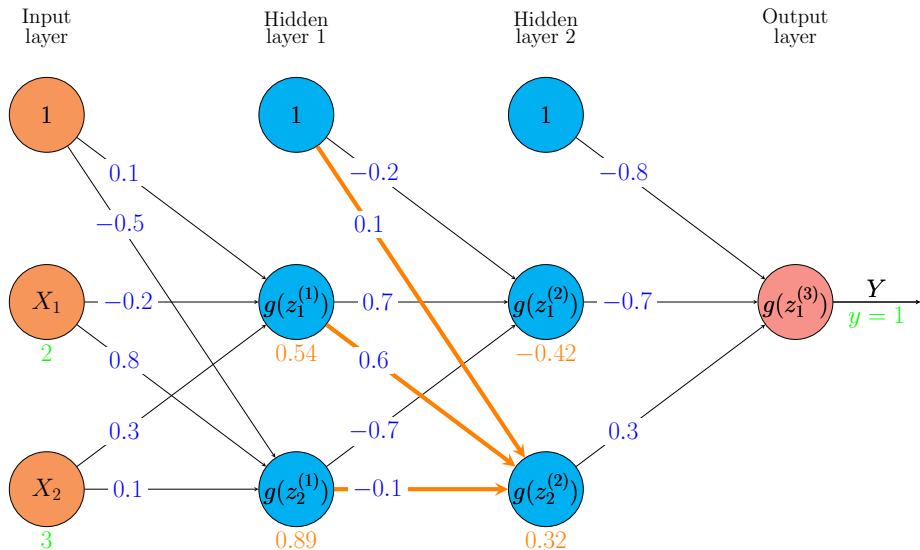
$$A_1^{(1)} = g(z_1^{(1)}) = \tanh \left(w_{10}^{(1)} + \sum_{j=1}^2 w_{1j}^{(1)} X_j \right) \approx 0.54$$



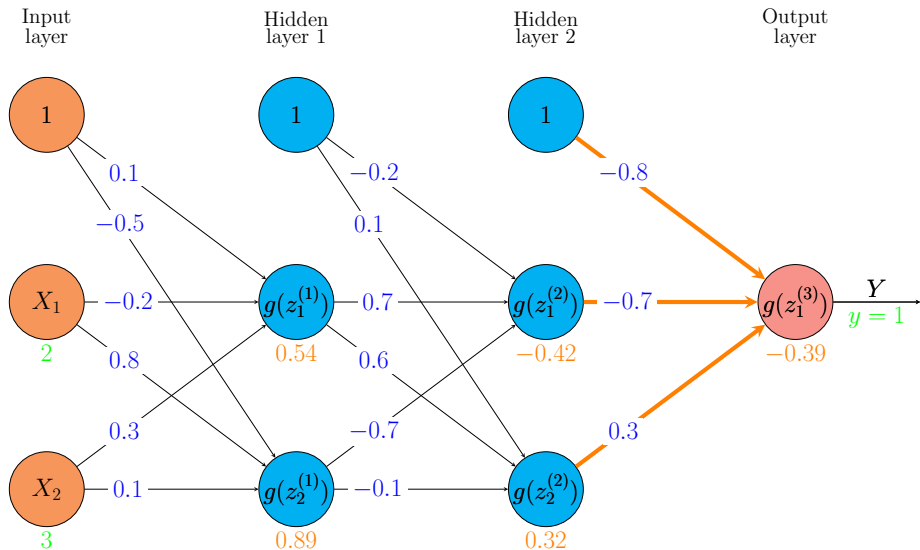
$$A_2^{(1)} = g(z_2^{(1)}) = \tanh \left(w_{20}^{(1)} + \sum_{j=1}^2 w_{2j}^{(1)} X_j \right) \approx 0.89$$



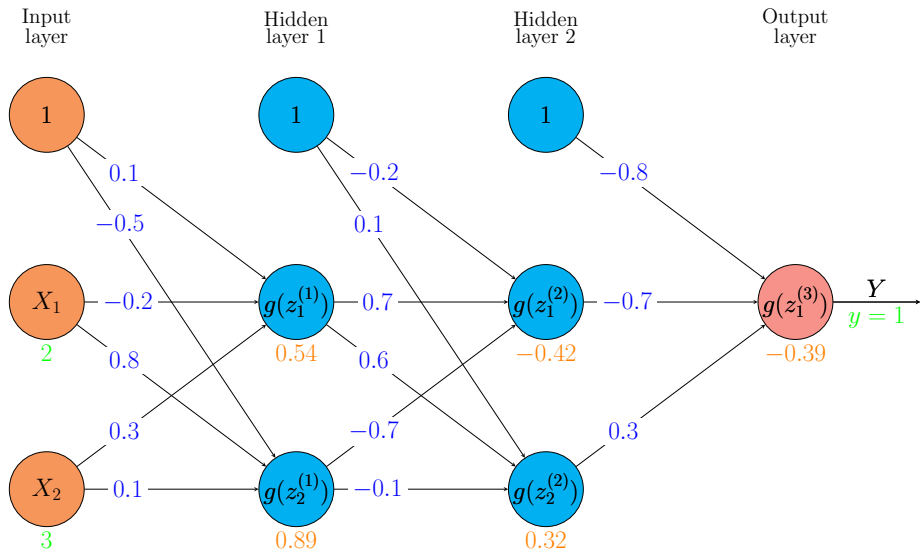
$$A_1^{(2)} = g(z_1^{(2)}) = \tanh \left(w_{10}^{(2)} + \sum_{j=1}^2 w_{1j}^{(2)} A_j^{(1)} \right) \approx -0.42$$



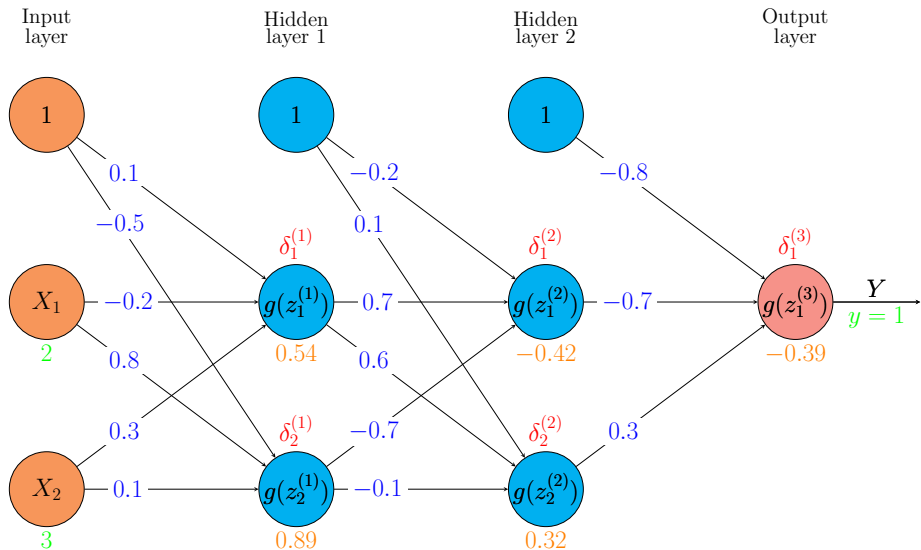
$$A_2^{(2)} = g(z_2^{(2)}) = \tanh \left(w_{20}^{(2)} + \sum_{j=1}^2 w_{2j}^{(2)} A_j^{(1)} \right) \approx 0.32$$



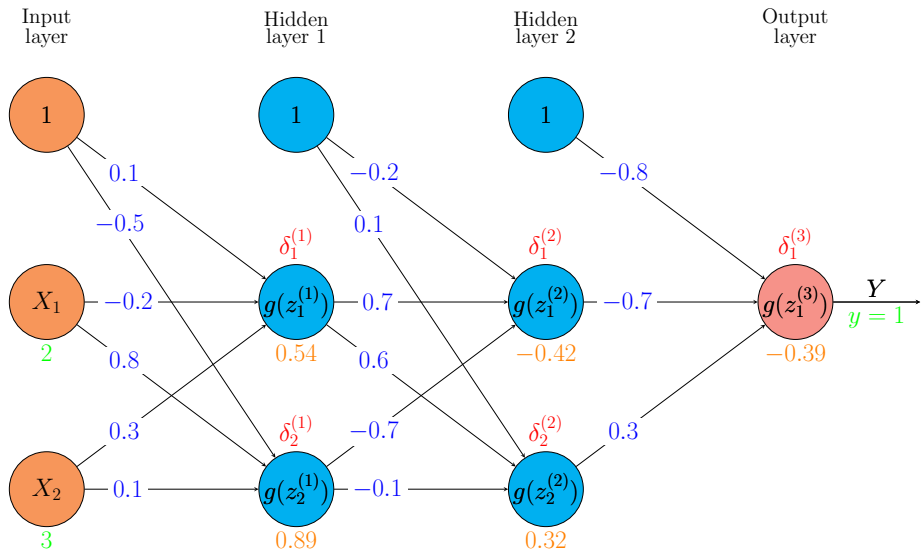
$$A_1^{(3)} = g(z_1^{(3)}) = \tanh \left(w_{10}^{(3)} + \sum_{j=1}^2 w_{1j}^{(3)} A_j^{(2)} \right) \approx -0.39$$



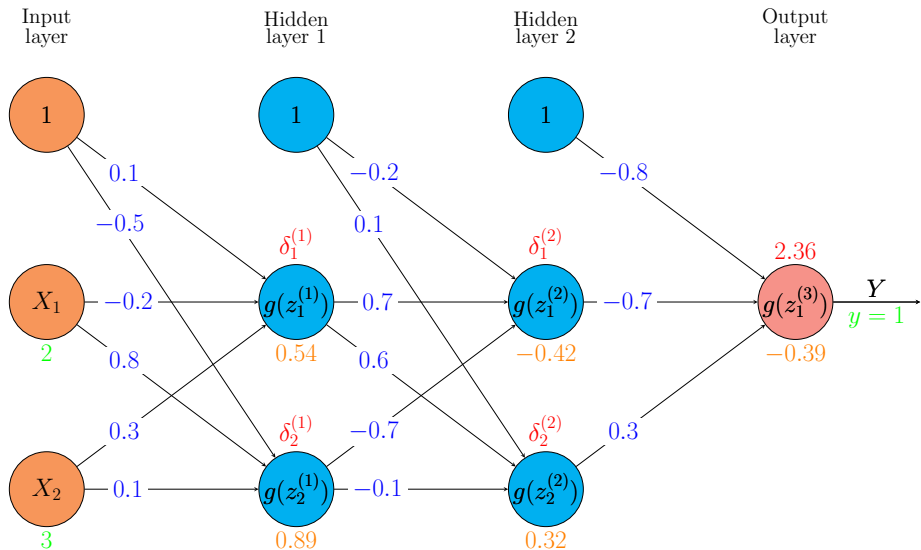
Now we need to compute each $\delta_k^{(\ell)}$ (backprop)



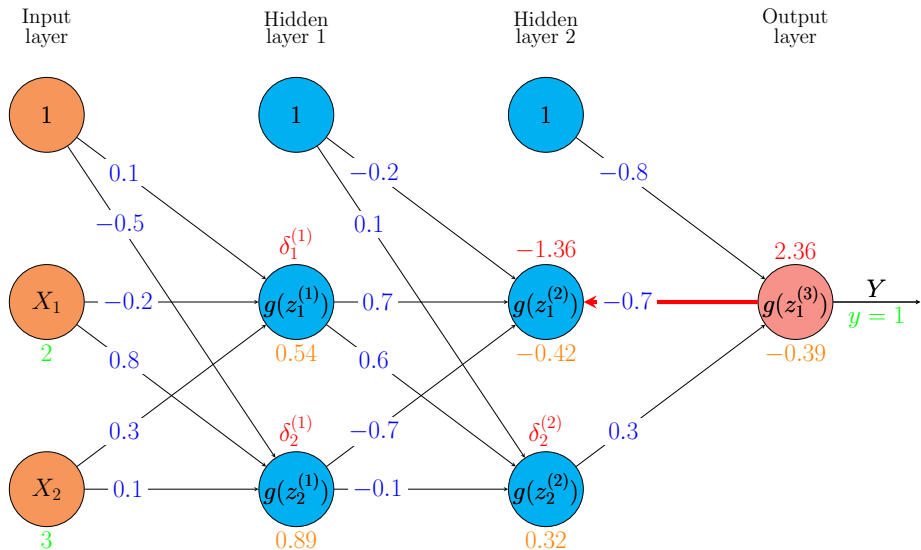
Now we need to compute each $\delta_k^{(\ell)}$ (backprop)



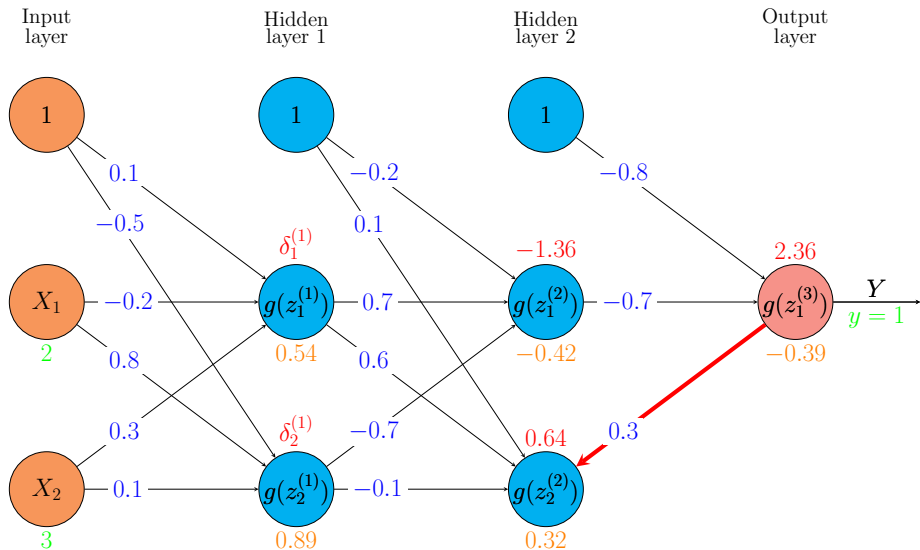
$$\delta_1^{(3)} = 2(y - A_1^{(3)})(1 - (A_1^{(3)})^2)$$



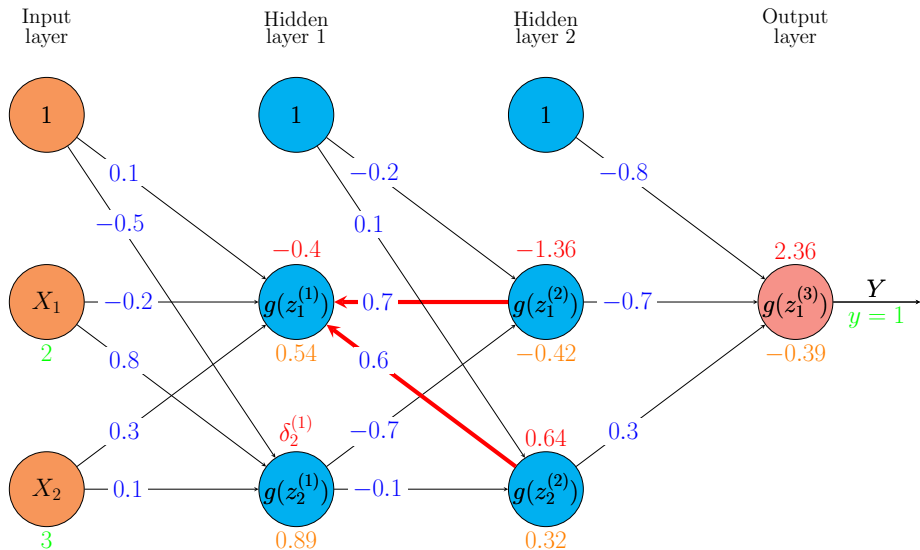
$$\delta_1^{(3)} = 2(y - A_1^{(3)})(1 - (A_1^{(3)})^2) = 2(1 - (-0.39))(1 - (-0.39)^2) \approx 2.36$$



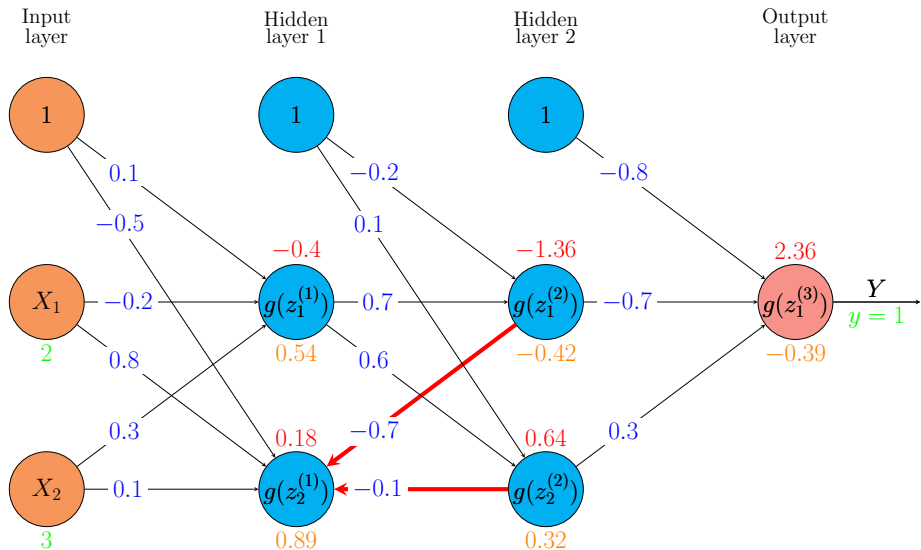
$$\delta_1^{(2)} = (1 - (A_1^{(2)})^2) \sum_{k=1}^1 \delta_k^{(3)} w_{k1}^{(3)} \approx -1.36$$



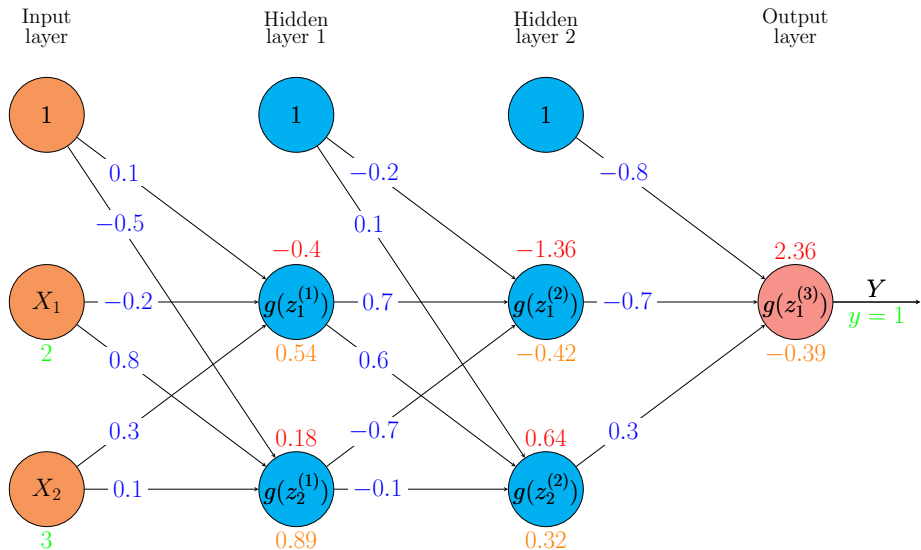
$$\delta_2^{(2)} = (1 - (A_2^{(2)})^2) \sum_{k=1}^1 \delta_k^{(3)} w_{k2}^{(3)} \approx 0.64$$



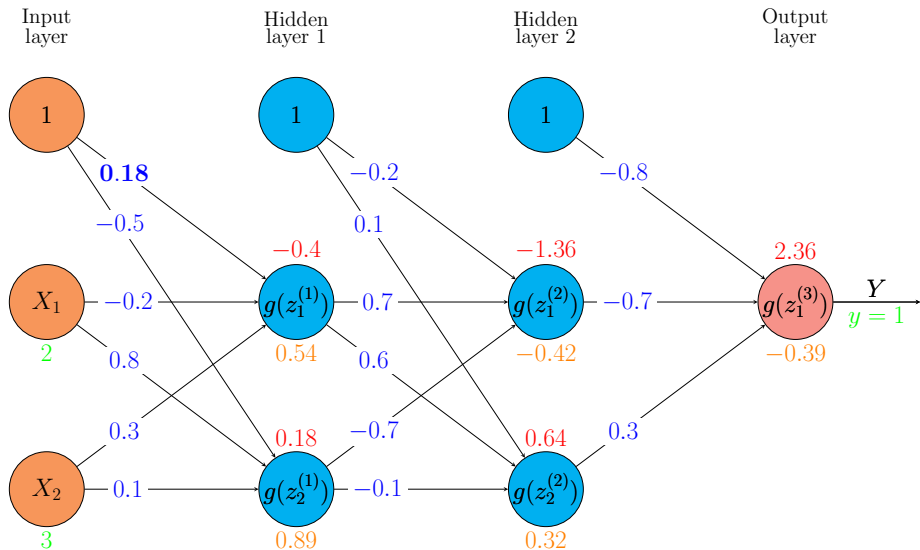
$$\delta_1^{(1)} = (1 - (A_1^{(1)})^2) \sum_{k=1}^2 \delta_k^{(2)} w_{k1}^{(2)} \approx -0.4$$



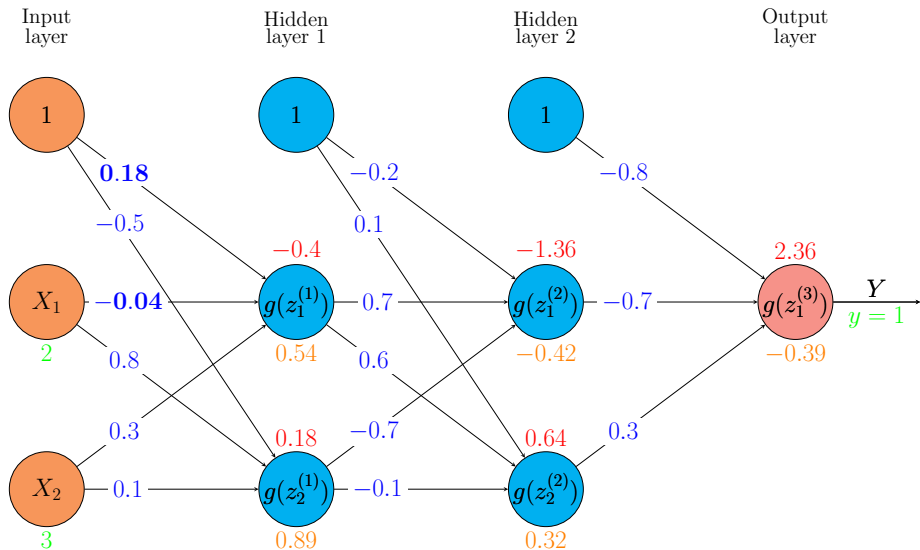
$$\delta_2^{(1)} = (1 - (A_2^{(1)})^2) \sum_{k=1}^2 \delta_k^{(2)} w_{k2}^{(2)} \approx 0.18$$



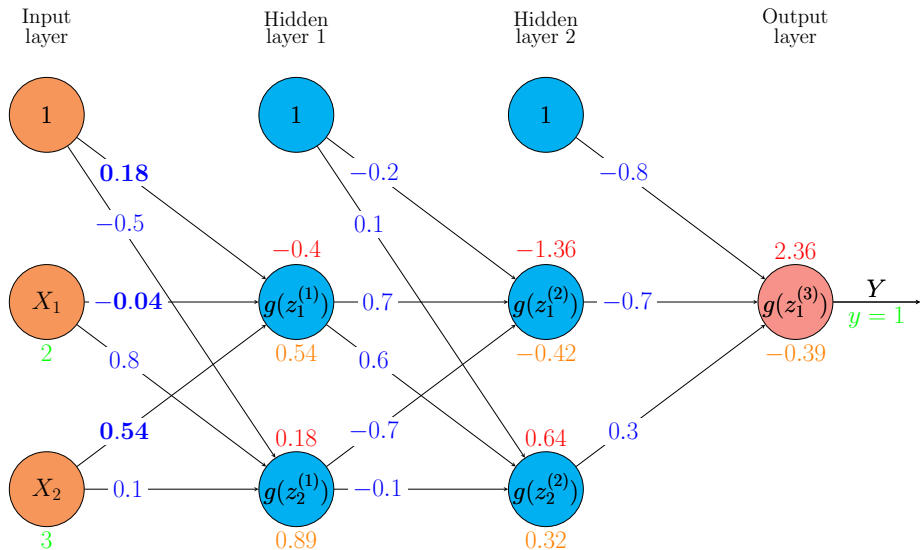
Finally, we update the weights (let's update with $\alpha = 0.1$)



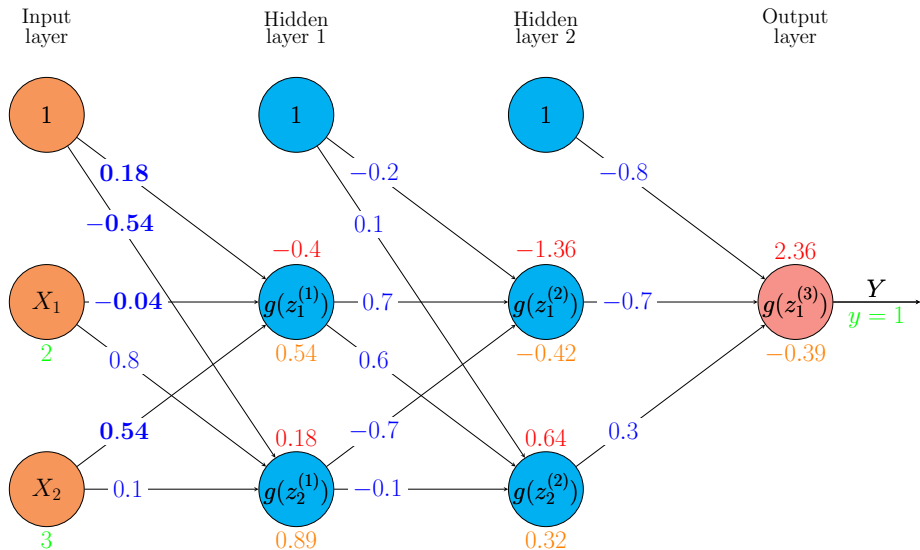
$$w_{10}^{(1)} \leftarrow w_{10}^{(1)} - \alpha \delta_1^{(1)} A_0^{(0)} = 0.1 - 0.1 \times -0.4 \times 1 \approx 0.18$$



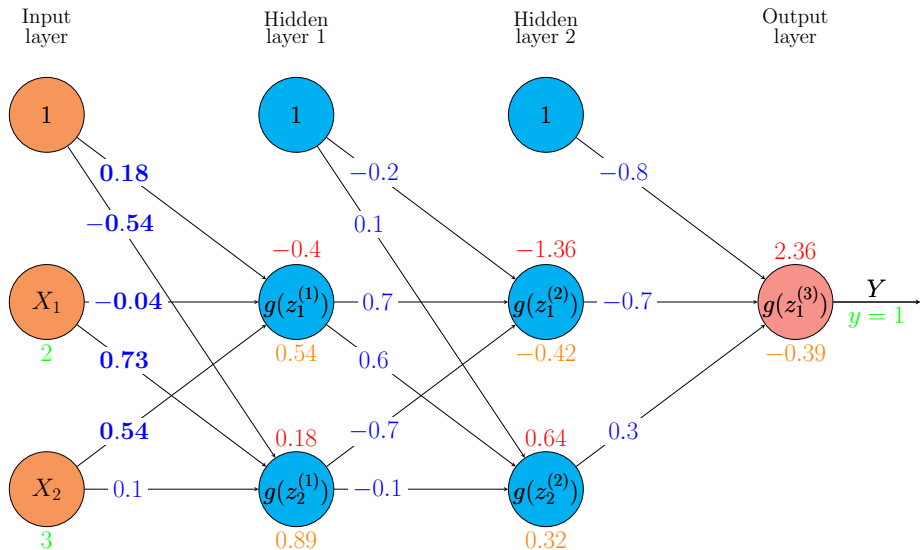
$$w_{11}^{(1)} \leftarrow w_{11}^{(1)} - \alpha \delta_1^{(1)} A_1^{(0)} = -0.2 - 0.1 \times -0.4 \times 2 \approx -0.04$$



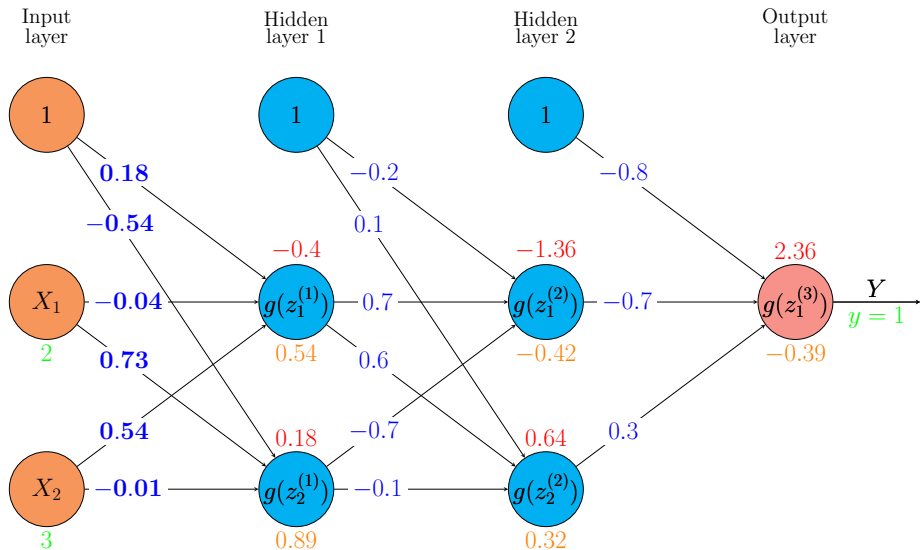
$$w_{12}^{(1)} \leftarrow w_{12}^{(1)} - \alpha \delta_1^{(1)} A_2^{(0)} = 0.3 - 0.1 \times -0.4 \times 3 \approx 0.54$$



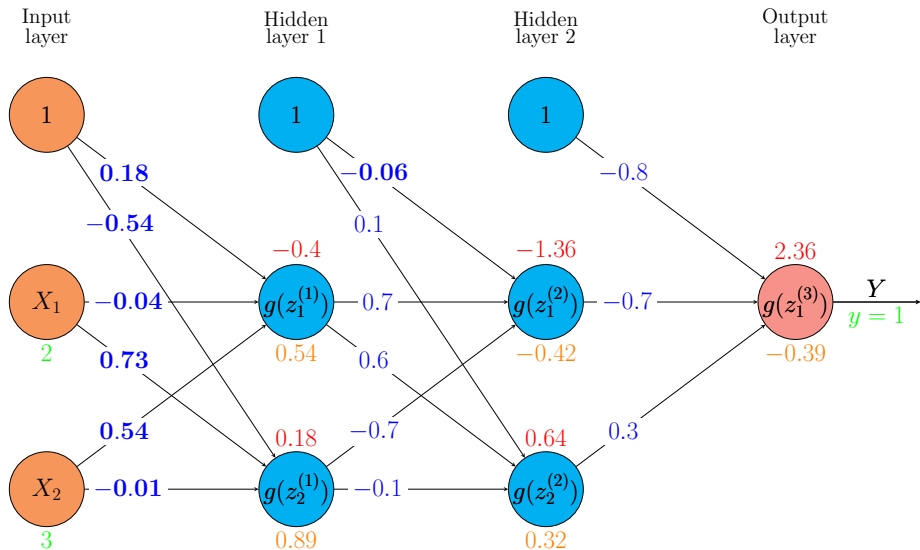
$$w_{20}^{(1)} \leftarrow w_{20}^{(1)} - \alpha \delta_2^{(1)} A_0^{(0)} = -0.5 - 0.1 \times 0.18 \times 1 \approx -0.54$$



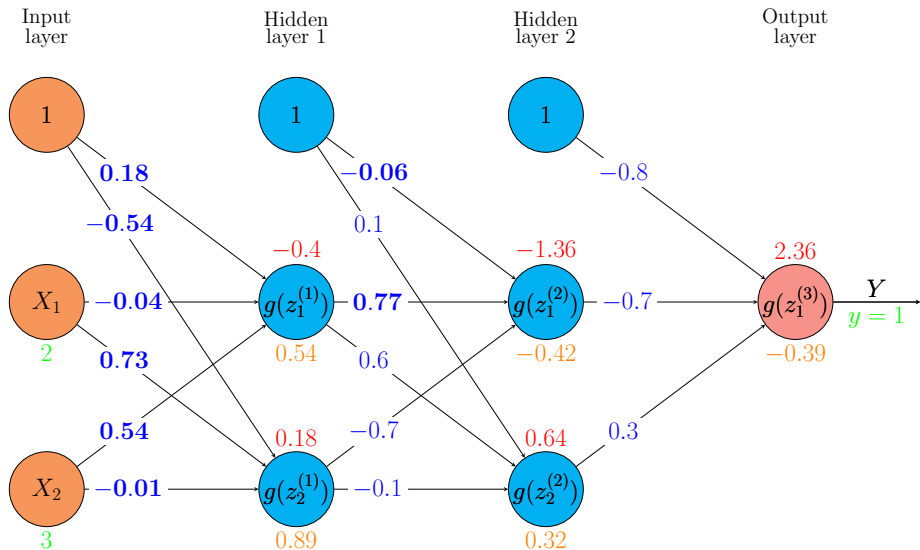
$$w_{21}^{(1)} \leftarrow w_{21}^{(1)} - \alpha \delta_2^{(1)} A_1^{(0)} = 0.8 - 0.1 \times 0.18 \times 2 \approx 0.73$$



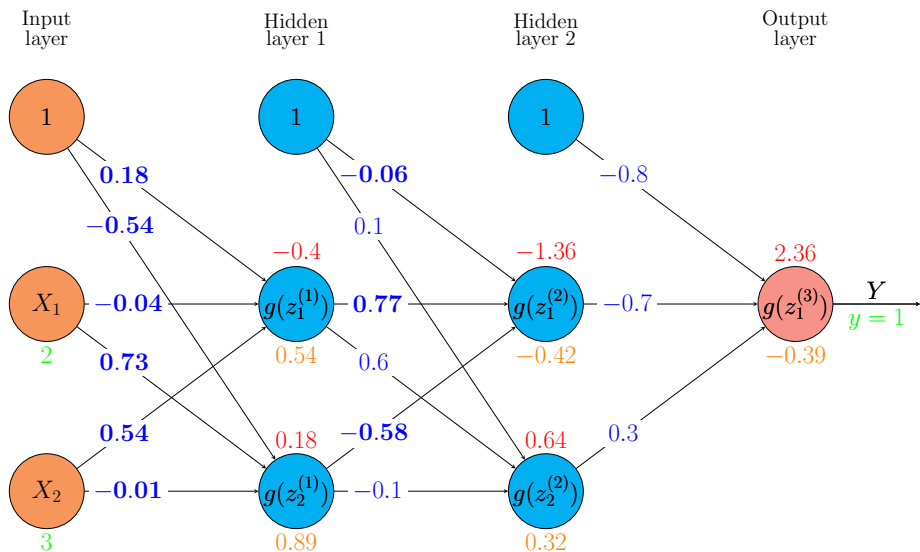
$$w_{22}^{(1)} \leftarrow w_{22}^{(1)} - \alpha \delta_2^{(1)} A_2^{(0)} = 0.1 - 0.1 \times 0.18 \times 3 \approx -0.01$$



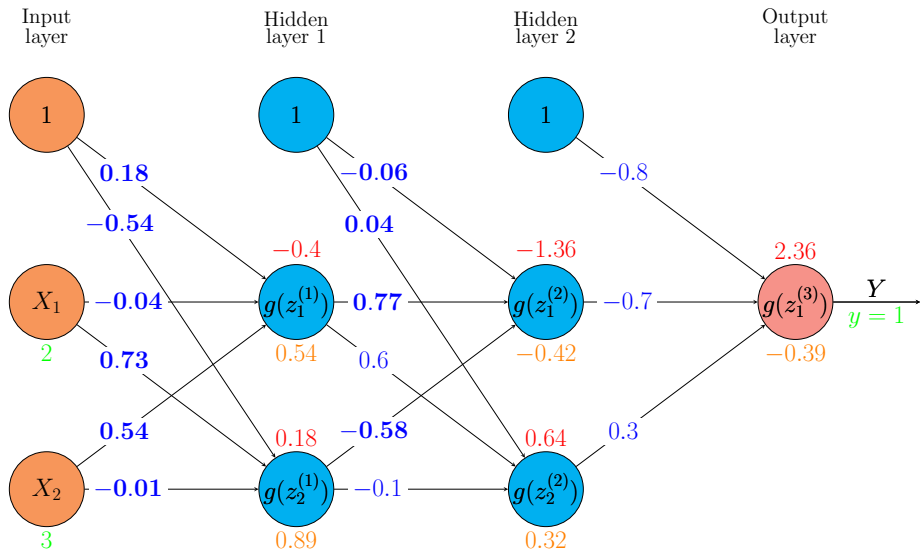
$$w_{10}^{(2)} \leftarrow w_{10}^{(2)} - \alpha \delta_1^{(2)} A_0^{(1)} = -0.2 - 0.1 \times -1.36 \times 1 \approx -0.06$$



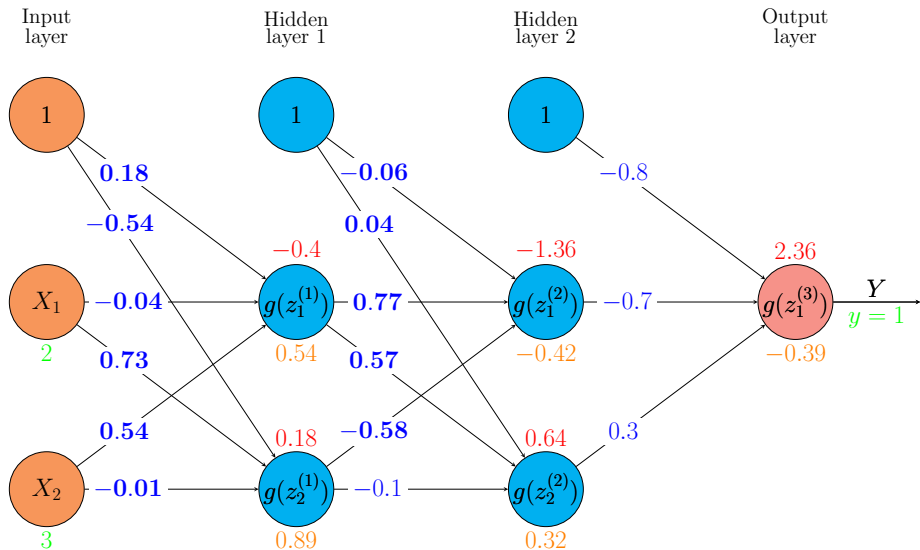
$$w_{11}^{(2)} \leftarrow w_{11}^{(2)} - \alpha \delta_1^{(2)} A_1^{(1)} = 0.7 - 0.1 \times -1.36 \times 0.54 \approx \mathbf{0.77}$$



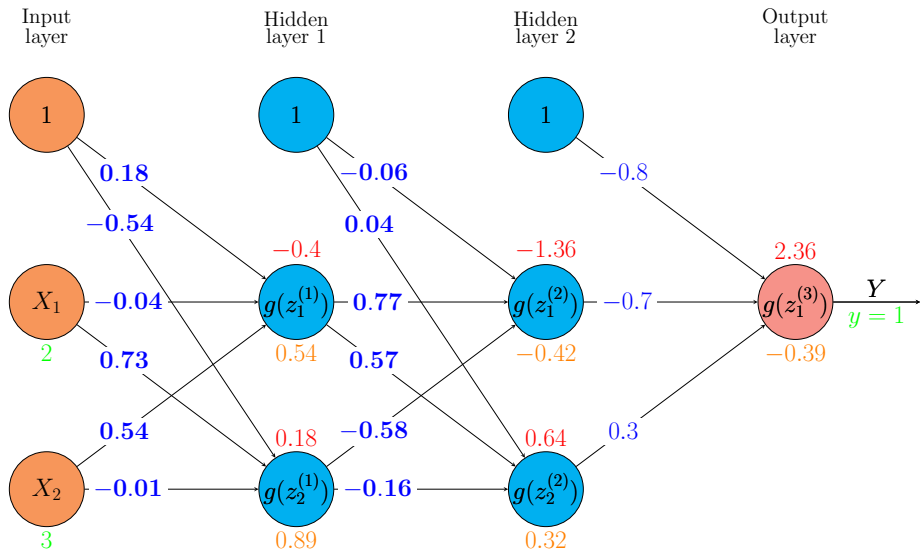
$$w_{12}^{(2)} \leftarrow w_{12}^{(2)} - \alpha \delta_1^{(2)} A_2^{(1)} = -0.7 - 0.1 \times -1.36 \times 0.89 \approx -0.58$$



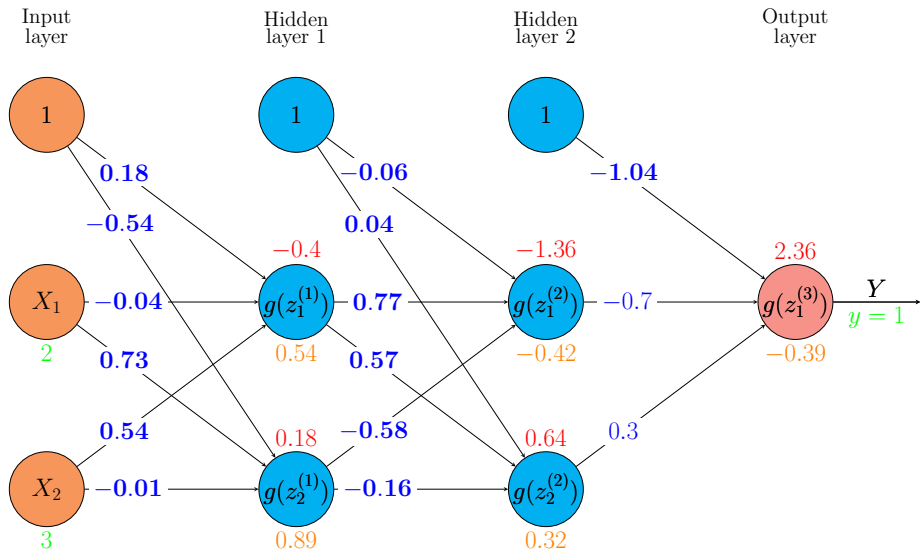
$$w_{20}^{(2)} \leftarrow w_{20}^{(2)} - \alpha \delta_2^{(2)} A_0^{(1)} = 0.1 - 0.1 \times 0.64 \times 1 \approx 0.04$$



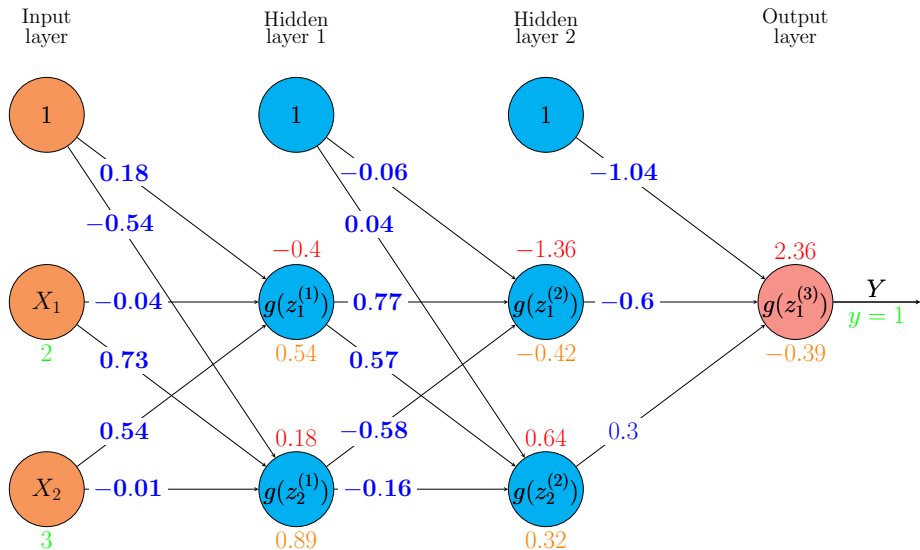
$$w_{21}^{(2)} \leftarrow w_{21}^{(2)} - \alpha \delta_2^{(2)} A_1^{(1)} = -0.7 - 0.1 \times 0.64 \times 0.54 \approx \mathbf{0.57}$$



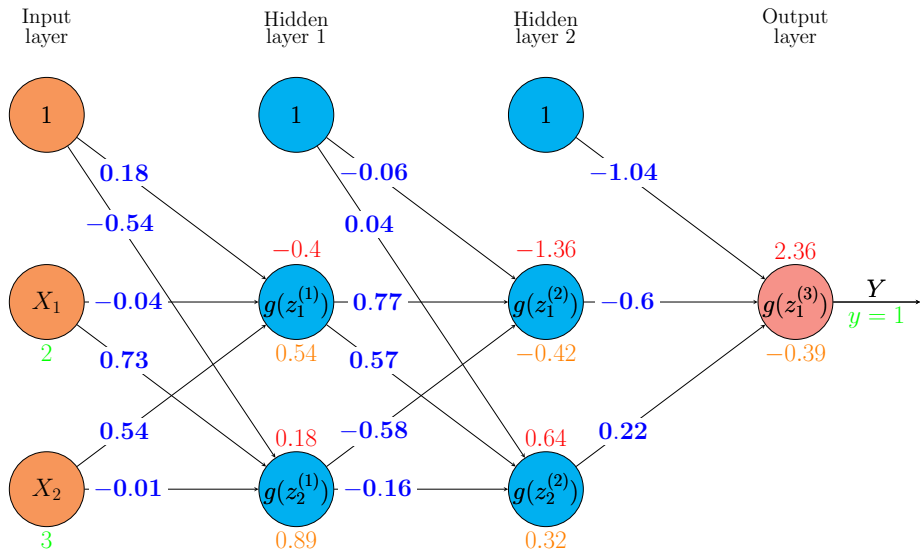
$$w_{22}^{(2)} \leftarrow w_{22}^{(2)} - \alpha \delta_2^{(2)} A_2^{(1)} = -0.1 - 0.1 \times 0.64 \times 0.89 \approx -0.16$$



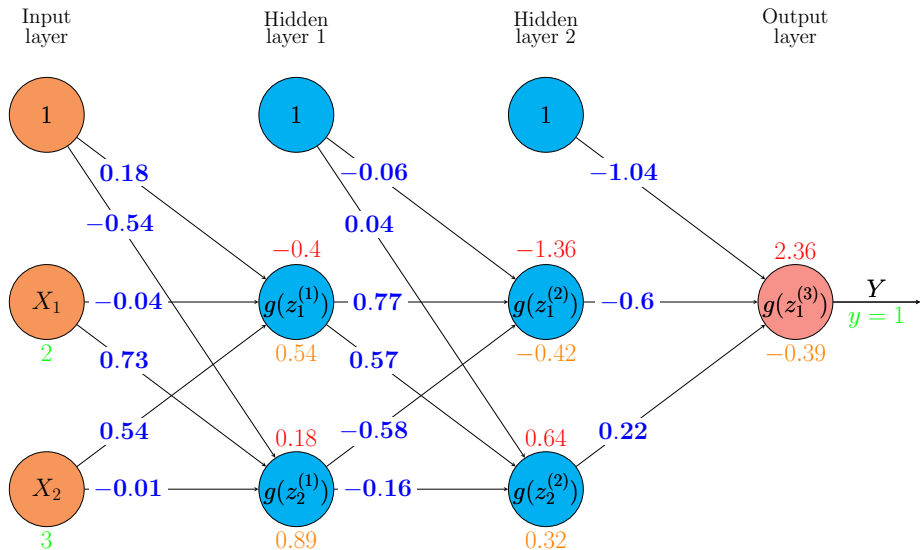
$$w_{10}^{(3)} \leftarrow w_{10}^{(3)} - \alpha \delta_1^{(3)} A_0^{(2)} = -0.8 - 0.1 \times 2.36 \times 1 \approx -1.04$$



$$w_{11}^{(3)} \leftarrow w_{11}^{(3)} - \alpha \delta_1^{(3)} A_1^{(2)} = -0.7 - 0.1 \times 2.36 \times -0.42 \approx -0.6$$



$$w_{12}^{(3)} \leftarrow w_{12}^{(3)} - \alpha \delta_1^{(3)} A_2^{(2)} = 0.3 - 0.1 \times 2.36 \times 0.33 \approx 0.22$$



End of iteration with observation (x, y)

```

STOCHASTIC-GRADIENT-DESCENT( $\{(x_1, y_1), \dots, (x_n, y_n)\}$ )
1   $w_{kj}^{(\ell)} \leftarrow$  a random value in  $[-1, 1]$ , for all  $j, k, \ell$ 

2  while stop criterion was not attained

3      do select  $(x_i, y_i)$ , one of the  $n$  observations

4          Compute forward all  $A_k^{(\ell)}$  with  $x_i$ 

5          Compute backward all  $\delta_k^{(\ell)}$  with  $y_i$ 

6          for each weight  $w_{kj}^{(\ell)}$ 

7              do  $w_{kj}^{(\ell)} \leftarrow w_{kj}^{(\ell)} - \alpha \cdot \delta_k^{(\ell)} \cdot A_j^{(\ell-1)}$ 

8  return  $\theta$            $\triangleright \theta = \{W^{(1)}, \dots, W^{(L)}\}$ 

```

Regularization

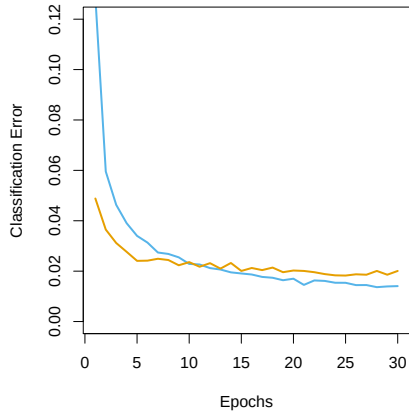
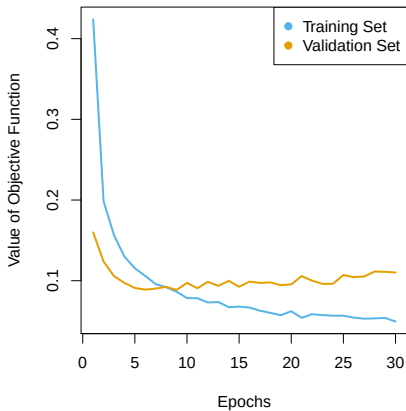
- ❖ We can use ridge regularization by augmenting the objective function (Eq. 10.25) with a penalty term:

$$R(\theta; \lambda) = \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2 + \underbrace{\lambda \sum_{j'} \theta_{j'}^2}_{\text{penalty term}}, \quad (10.31^*)$$

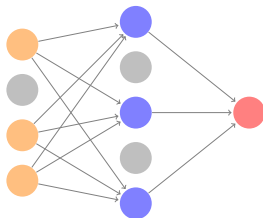
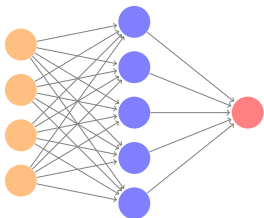
where $\theta_{j'}$ is a weight $w_{kj}^{(\ell)}$ and λ is a hyperparameter

- ❖ We can also use different values of λ for different $w^{(\ell)}$
- ❖ Another alternatives are *early stopping* and *dropout*

Early stopping on MNIST data



Dropout learning



Left: a fully connected network

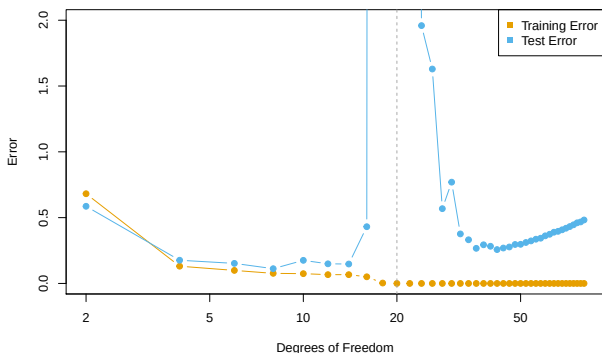
Right: network with dropout units in the input and hidden layer

Interpolation and double descent

- ❖ Error as a function of model flexibility is often U-shaped
- ❖ This is due the bias-variance trade-off
- ❖ In previous lectures, we saw that interpolate data points is a bad thing (why?)
- ❖ However, there are some exceptions, such as in the *double descent* phenomenon

Double descent in one-dimensional natural spline **

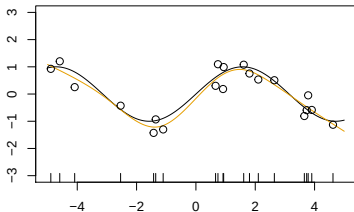
(**Natural splines are nonlinear models that generalize linear models of ISLP Chapters 3-4. They are featured in Chapter 7)



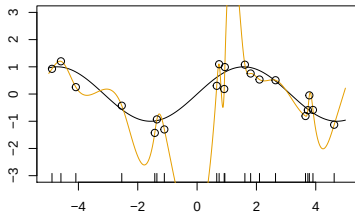
- ❖ Data generator was $Y = \sin(X) + \epsilon$, with $X \sim U[-5, 5]$ and $\epsilon \sim N(0, 0.9)$
- ❖ $n = 20$ data points were generated
- ❖ Dashed line shows the “interpolation threshold”

Fitted functions (orange) against true function (black)

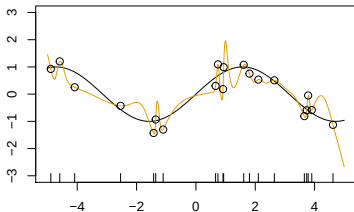
8 Degrees of Freedom



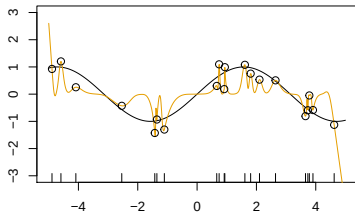
20 Degrees of Freedom



42 Degrees of Freedom



80 Degrees of Freedom



Double descent and neural networks

- ❖ In the previous example, for models with ≥ 20 degrees of freedom (d) the training error is zero
- ❖ For $d = 20$ there is only one way to interpolate data, while for $d > 20$ there are infinite ones
- ❖ For $d > 20$, *minimum-norm solutions* (i.e., interpolations that minimizes $\sum_{j=1}^P \hat{\beta}_j^2$) were chosen, yielding a “smoother” model
- ❖ Similar effect might occur with neural networks
- ❖ Might be acceptable if signal-to-noise ratio is high (i.e., data points are close to the true curve)